

## Bài 6 – Tiếp chương 4 CÁC THUẬT TOÁN SẮP XẾP

69it, 9/02/2025.

### IV. SẮP XẾP KIỂU CHÈN

Thuật toán này dùng trong trường hợp khi tạo một danh sách, vừa nhập phần tử vào vừa so sánh lựa chọn vị trí để đặt phần tử theo đúng trật tự yêu cầu (tăng dần hay giảm dần giá trị) chứ không phải nhập cả dãy số vào rồi mới sắp xếp.

Ví dụ, có dãy số 67,33,21,84,49. Có thể nhập lần lượt các số vào từ bàn phím, mỗi lần nhập số mới cần so sánh với các số đã nhập để sắp xếp sao cho dãy đã nhập có giá trị tăng dần; cũng có thể nhập hết các số sau đó lần lượt so sánh các số từ đầu đến cuối để sắp xếp sao cho dãy có giá trị tăng dần. Ở đây tiến hành theo cách nhập số đến đâu sắp xếp chèn vào đến đấy.

Cụ thể : khai báo dãy số  $x[5]$ ;

Bước 1. cho  $i=1$ , nhập  $x[1] = 67$

| $x[1]$ |
|--------|
| 67     |

Bước 2.  $i=2$ ;

Thêm biến phụ  $r$  để lưu số mới nhập; nhập  $r = 33$

So sánh số mới nhập  $r=33$  với số 67 trước đó, vì  $33 < 67$  nên đưa số 33 ra trước số 67.

Đoạn thuật toán của việc “So sánh số mới nhập 33 với số 67 trước đó, vì  $33 < 67$  nên đưa số 33 ra trước số 67” được diễn đạt như sau :

\*Thêm biến  $j$  để chỉ vị trí phần tử đứng trước phần tử mới nhập,  $j = i-1=1$ ,  $x[j]=67$ .

\*So sánh  $r$  với  $x[j]$ , nếu  $(r < x[j])$  là TRUE thì  $x[j+1] = x[j]$ ;  $x[j] = r$ ; FALSE thì  $x[j+1] = r$ .

Câu lệnh if là :  $\text{if}(r < x[j]) \{x[j+1]=x[j]; x[j]=r\}$  else  $x[j+1]=r$  ;

Kết quả :

| $x[1]$ | $x[2]$ |
|--------|--------|
| 33     | 67     |

Bước 3. Tăng  $i=3$ ; nhập số mới  $r = 21$ ;  $j = i-1=2$ ,  $x[j]=67$ .

Để sắp xếp đúng trật tự tăng dần, cần so sánh số  $r=21$  với 67, đưa 21 về trước 67:

Thuật toán của việc vừa làm này được diễn đạt là :

so sánh  $r=21$  với  $x[j]=67$ , nếu  $(r < x[j])$  là TRUE thì  $x[j+1] = x[j]$ ;  $x[j] = r$  ; nếu FALSE thì  $x[j+1] = r$ . Kết quả :

| $x[1]$ | $x[2]$ | $x[3]$ |
|--------|--------|--------|
| 33     | 21     | 67     |

Vì  $r = 21 < 33$  mà còn đứng sau 33 nên cần phải sắp xếp tiếp để đưa 21 về trước 33 , hiện tại  $r$  đang ở vị trí  $x[2]$ .

Để làm tiếp, phải giảm  $j = j-1$ ; ( $j$  mới  $=1$ ) ,  $x[j]=33$ .

Lập lại :

So sánh  $r=21$  với  $x[j=1]=33$ , nếu  $(r < x[j])$  là TRUE thì  $x[j+1] = x[j]$ ;  $x[j] = r$ ; nếu FALSE thì  $x[j+1] = r$ . Kết quả :

| $x[1]$ | $x[2]$ | $x[3]$ |
|--------|--------|--------|
| 21     | 33     | 67     |

Bước này đến đây phải dừng, nghĩa là khi  $j=1$ , đã về đến đầu dãy số là phải kết thúc.

**Bước 4.** Tăng  $i = 4$ ; nhập số mới  $r = 84$ ;  $j = i-1 = 3$ ,  $x[j=3]=67$ .

Lặp lại thủ tục như trên : so sánh  $r=84$  với  $x[j]=67$ , nếu  $(r < x[j])$  là TRUE thì  $x[j+1] = x[j]$ ;  $x[j] = r$ ; nếu FALSE thì  $x[j+1] = r$ . Kết quả  $x[4]=84$

| $x[1]$ | $x[2]$ | $x[3]$ | $x[4]$ |
|--------|--------|--------|--------|
| 21     | 33     | 67     | 84     |

**Bước 5.** Tăng  $i = 5$ , nhập số mới  $r = 49$ ;  $j=i-1=4$ ,  $x[j=4]=84$ .

Lặp lại thủ tục như trên, so sánh số mới  $r=49$  với  $x[j=4]=84$ .

nếu  $(r < x[j])$  là TRUE thì  $x[j+1] = x[j]$ ;  $x[j] = r$ ; nếu FALSE thì  $x[j+1] = r$ . Lệnh kiểm tra cho TRUE nên kết quả là:

| $x[1]$ | $x[2]$ | $x[3]$ | $x[4]$ | $x[5]$ |
|--------|--------|--------|--------|--------|
| 21     | 33     | 67     | 49     | 84     |

Vì  $r = 49 < 67$  mà còn đứng sau 67 nên cần phải sắp xếp tiếp để đưa 49 về trước 67, hiện tại  $r$  đang ở vị trí  $x[4]$ .

Để làm tiếp, phải giảm  $j=j-1$ ; ( $j$  mới = 3),  $x[3] = 67$ .

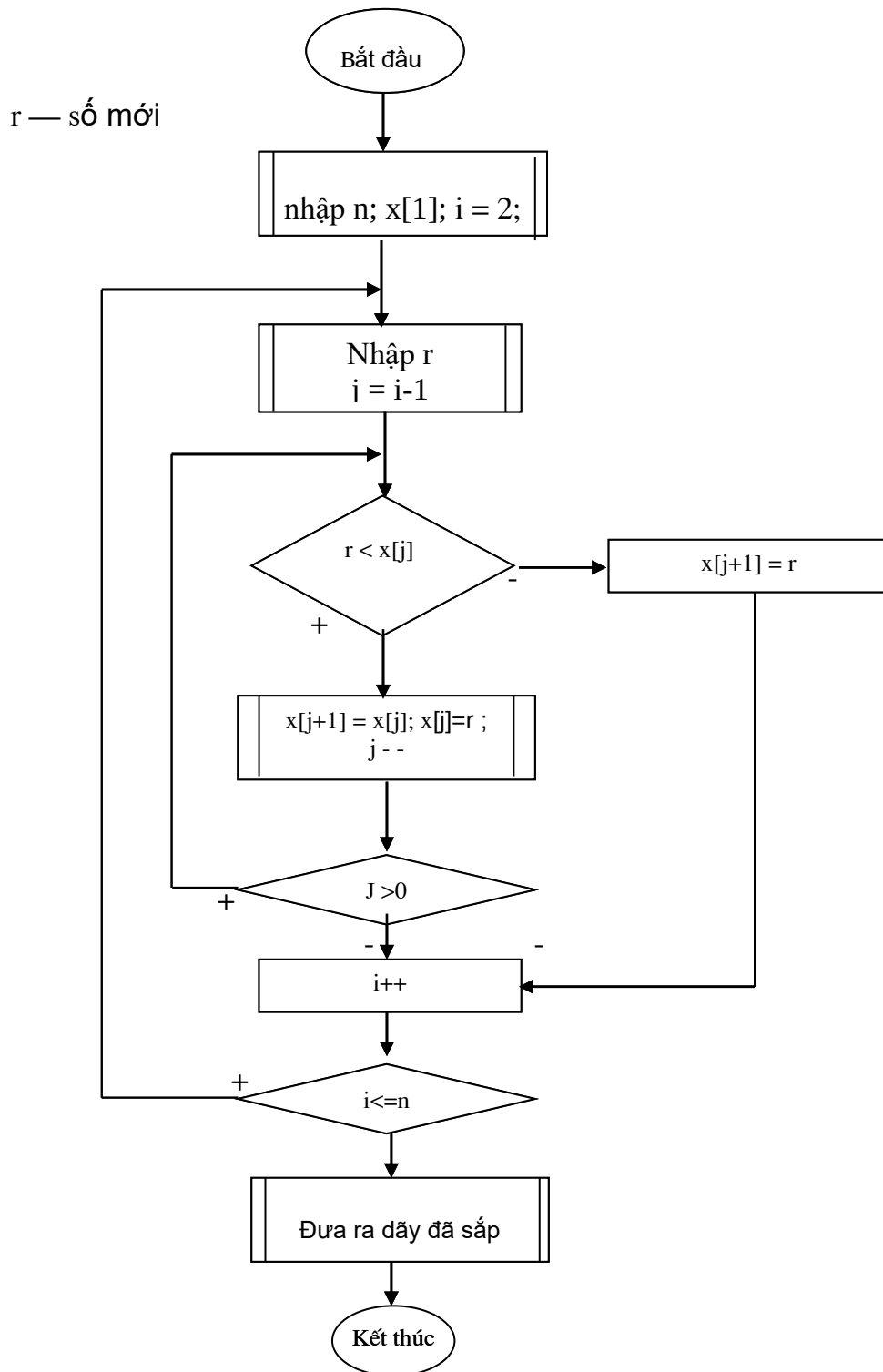
Lặp lại : so sánh  $r=49$  với  $x[j=3]=67$ , nếu  $r < x[j]$  TRUE thì  $x[j+1] = x[j]$ ;  $x[j] = r$ ; nếu FALSE thì  $x[j+1] = r$ , Lệnh kiểm tra cho TRUE nên kết quả :

|    |    |    |    |    |
|----|----|----|----|----|
| 21 | 33 | 49 | 67 | 84 |
|----|----|----|----|----|

Hiện tại số mới  $r$  đang ở vị trí  $x[j=3]$ , cần phải lặp lại đoạn tính lặp trên cho đến khi  $r > x[j]$  hoặc đến khi  $j = 1$  thì mới khẳng định được đã xét hết dãy số.

Vì bài toán chỉ đặt ra nhập 5 số ( $n=5$ ) nên đến đây sẽ có lệnh kiểm tra kết thúc bước lặp theo  $i$ , khi  $i=5$  thì thoát ra.

**Nhận xét :** theo thuật toán trên đây, trước khi nhập số mới vào thì các số trước đó đã được sắp xếp tăng dần, vì vậy nếu số mới nhập  $>$  số vừa nhập trước đó thì chỉ việc đặt số mới nhập đứng sau số vừa nhập trước đó; nếu ngược lại thì mới so sánh số mới nhập với các số đã nhập trước đó và thế chỗ dịch chuyển lần lượt về hướng đầu dãy cho đến khi tìm đúng vị trí cần chèn vào. Qua trình này lặp lại nhiều nhất khi số mới nhập phải đưa về vị trí đầu dãy  $x[1]$ .



Bài tập : viết tiếp các dòng lệnh để hoàn thiện chương trình sắp xếp chèn :

```

int main()
{
    int i=1,j,n;
    float r,x[15];
    cout << "\n\n    SAP XEP CHEN " << endl;

```

```

cout << "\n  Nhap N : " ;
.....
.....
.....

cout<<"\n\n  DAY SO KET QUA : \n"<<endl;
for (i=1;i<=n;i++)
{ cout << "    "<<x[i]<< ", ";
}
cout<<"\n\n";
system("pause");
return 0;
}

```

## CHƯƠNG 5. CÁC THUẬT TOÁN TÌM KIẾM VÀ ĐỆ QUY

### 5.1. TÌM KIẾM

#### 5.1.1. Tìm phần tử trong một danh sách tuyến tính đã được sắp thứ tự.

Tìm kiếm tuyến tính là bắt đầu từ phần tử đầu tiên, tìm một cách tuần tự trong danh sách cho đến khi tìm ra phần tử đã cho hoặc đạt đến cuối danh sách. Kiểu danh sách là mảng các phần tử Kiểu\_Ptu.

Ví dụ : Cho danh sách lớp có n người, có các cột mã sinh viên, họ tên, điểm trung bình học kỳ. Vẽ sơ đồ thuật toán và viết chương trình tìm và lập ra danh sách những người có điểm  $\geq 8$ , danh sách những người có điểm cao nhất lớp, danh sách mới này cũng có các cột mã sinh viên, họ tên, điểm trung bình học kỳ.

Cách làm :

- Tạo một danh sách lớp - DS1 có n dòng, có các cột

| Mã sinh viên | Họ tên | Điểm tb |
|--------------|--------|---------|
|--------------|--------|---------|

- Tạo một danh sách DS2 - những người có điểm  $\geq 8$  có n dòng, có các cột

| Mã sinh viên | Họ tên | Điểm tb |
|--------------|--------|---------|
|--------------|--------|---------|

và một danh sách những người có điểm cao nhất lớp – DS3 cũng có n dòng, 2 cột :

| Mã sinh viên | Họ tên |
|--------------|--------|
|--------------|--------|

- Nhập số phần tử n;
- Nhập danh sách DS1;
- Tìm điểm lớn nhất Max;

- Tạo DS2 : lần lượt kiểm tra so sánh điểm của từng sinh viên, ai có điểm  $\geq 8$  thì đưa Mã sinh viên, họ tên, điểm tb học kỳ của người đó sang DS2.
- Tạo DS3 : lần lượt kiểm tra so sánh điểm của từng sinh viên, ai có điểm bằng điểm Max thì đưa Mã sinh viên và họ tên người đó sang DS3.
- Hiện lại DS1, DS2, DS3 .

Hướng dẫn : dùng chỉ số i, j, k lần lượt làm chỉ số trong các danh sách DS1, DS2, DS3.

Khởi gán  $i=1, j=0, k=0$ .

Đặt tên các biến :

Trong DS1 : masv1[] – mã sinh viên; hoten1[] – họ tên ,tb1[] – điểm trung bình học kỳ,

trong DS2 : masv2[] – mã sinh viên; hoten2[] – họ tên ,tb2[] – điểm trung bình học kỳ,

Trong DS3 : masv3[] – mã sinh viên; hoten3[] – họ tên.

Max – điểm lớn nhất.

*Phần tạo DS2 :*

Cho  $j=0$ ; Cho  $i=1$  đến  $n$  thực hiện :

Nếu  $(tb1[i] \geq 8)$  thì  $\{j++; masv2[j]=masv1[i]; hoten2[j]=hoten1[i]; tb2[j]=tb1[i];\}$

*Phần tạo DS3 :*

*Phần tìm điểm Max :*

$Max = tb1[1]$ ; Cho  $i=2$  đến  $n$  thực hiện : nếu  $(tb1[i] > Max)$  thì  $Max=tb1[i]$ ;

*Tạo DS3 :* Cho  $k=0$ ; Cho  $i=1$  đến  $n$  thực hiện :

Nếu  $(tb1[i] = Max)$  thì  $\{k++; masv3[k]=masv1[i]; hoten3[k]=hoten1[i];\}$

Bài tập : viết tiếp các dòng lệnh để hoàn thành chương trình :

```
#include <iostream>
```

```
#include <string>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main()
```

```
{ int i,j,k,n;
```

```
string masv1[50],hoten1[50], masv2[50],hoten2[50], masv3[50],hoten3[50];
```

```
float tb1[50], tb2[50],tb3[50];
```

```
cout<<"  Nhap Du lieu : \n";
```

```
cout<<"  Nhap N = ";cin>>n;
```

```
for(i=1;i<=n;i++)
```

```
{
```

```
.....
```

```
.....
```

```
}
```

```
//Lập danh sách sv có dtb>=8.
j=0;
//Lập danh sách sv có điểm Max
float Max = 0;

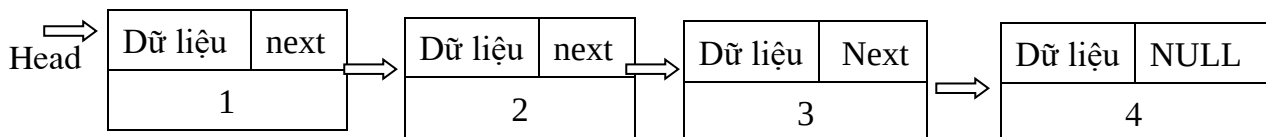
//Hiện các danh sách

system("pause");
return 0;
}
```

### 5.1.2. Tìm kiếm trong một danh sách liên kết

Tìm kiếm phần tử trong danh sách liên kết đơn cũng là duyệt danh sách từ đầu cho đến khi tìm thấy phần tử hoặc đến hết danh sách (`node == NULL`) mà không thấy, nếu tìm thấy, sẽ trả về node đó.

Ta có sơ đồ :



Nhập dữ liệu để tìm ->a, sau đó lần lượt tìm các nút, so sánh dữ liệu `node.du_lieu` với a, nếu bằng(==) thì đưa ra thông báo “đã tìm thấy”, nếu đến hết danh sách không tìm thấy thì đưa ra tb “không tìm thấy”

```
Node* TimNut(Node* list, int a)
{
    Node* node = list;
    while (node != NULL && node->du_lieu != a)
        node = node->next;
    if(node != NULL){
        cout<<" Da tim thay nut co du lieu ="<a<<" \n";
        return node; }
    cout<<" Khong tim thay nut co du lieu ="<a<<" \n";
    return NULL;
}
```

Thêm hàm này vào chương trình E:\Nam20\_21\dslk24\_4\_21 đã có khi học dslkết đơn :

```
#include<iostream>
#include <iomanip>
using namespace std;

struct Node{ int du_lieu;
```

```

        Node* next; };

Node* KhoitaoNode(int du_lieu0){ Node* node = new Node;
                                node->du_lieu = du_lieu0;
                                node->next = NULL;
                                return node;
                                }

void HienDS(Node* list){//printList(node *l){
    Node *p = list;
    cout<<setw(30);
    while (p != NULL){ cout << p->du_lieu <<setw(5);p = p->next; }
    cout<<"\n\n";
    system("pause"); }

Node* ThemvaoCuoi(Node* node, int a)
{ Node* p = KhoitaoNode(a);
  node->next = p;
  return p;
}

Node* TimNut(Node* list, int a)
{ Node* node = list;
  while (node != NULL && node->du_lieu != a) node = node->next;
  if(node != NULL){cout<<" Da tim thay nut co du lieu ="<<a<<" \n";
                  return node; }
  cout<<" Khong tim thay nut co du lieu ="<<a<<" \n";
  return NULL;
}

int main(){int n, a;
  cout<<"\n\n  Nhap so phan tu cua dslk :";
  cin >> n;
  cout<<"\n\n  Nhap du lieu phan tu dau tien cua dslk :";
  cin >> a;
  Node* list = KhoitaoNode(a);
  Node* node = list;
  for (int i = 1; i < n; i++){
      cout<<"\n  Nhap du lieu phan tu tiep theo cua dslk :";
      cin >> a; node = ThemvaoCuoi(node, a);
  }

  cout<<"\n\n  Hien du lieu cua dslk : \n\n";
  HienDS(list);
  cout<<"  Ban muon tim nut co du lieu = ?";cin>>a;
}

```

```

    TimNut(list,a);
    system("pause");
    return 0;
}

```

**Bài tập :** từ phần đã được giải thích, vẽ lại sơ đồ thuật toán của tìm kiếm phần tử trong danh sách liên kết

### 5.1.3. Tìm kiếm nhị phân trong tập dữ liệu đã được sắp thứ tự

#### 1) Ý tưởng

Tìm kiếm nhị phân đã được nghiên cứu từ lâu trong toán học tính toán.

Muốn tìm một phần tử trong một tập  $\Delta$  đã được sắp thứ tự, lần thứ nhất ta chia đôi tập  $\Delta$  thành  $A_1, A_2$  ; Lúc đầu tìm phần tử trong  $A_1$ , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi  $A_2$  thành  $B_1, B_2$ ; tiếp tục tìm phần tử trong  $B_1$ , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi  $B_2$  thành  $C_1, C_2$ ... quá trình tiếp tục cho đến khi tìm được phần tử.

Theo cách chia đôi : tập  $A_i$  có lực lượng =  $1/2 \Delta \rightarrow 1/2^1$

$B_i$  có lực lượng =  $1/4 \Delta \rightarrow 1/2^2$

$C_i$  có lực lượng =  $1/8 \Delta \rightarrow 1/2^3$

Ở bước thứ  $n$ , tập được chia đôi có lực lượng  $\rightarrow 1/2^n$

Với  $n=10$  thì  $2^{10}=1024$ , nghĩa là ta đã xét được trong tập  $\Delta$  có hơn một nghìn phần tử.

#### 2) Ứng dụng

Tìm phần tử trong danh sách  $x_1, x_2, \dots, x_n$ . Đặt biến *found* sẽ có giá trị đúng khi tìm được và *mid* (midle – điểm giữa) là vị trí điểm chia đôi các phần để tìm .

1. found = FALSE; first = 1; last = n ;//first – điểm đầu, last- điểm cuối

2. while (( first <=last )&&(! found ))

{ mid = first+ (last - first)/2; //ví dụ first=10, last=20, mid= 10+(20-10)/2=15

if (item < x[mid] ) last = mid -1 ; //item – phần tử cần tìm , xét x[10]- x[14]

else

{ if(item > x[mid]) first = mid+1; // xét x[16]- x[20]

else found = TRUE

}

}

**Bài tập :** vẽ sơ đồ khối thuật toán và viết chương trình C++ tìm phần tử trong một tập các phần tử đã được sắp thứ tự theo phương pháp chia đôi(nhị phân)



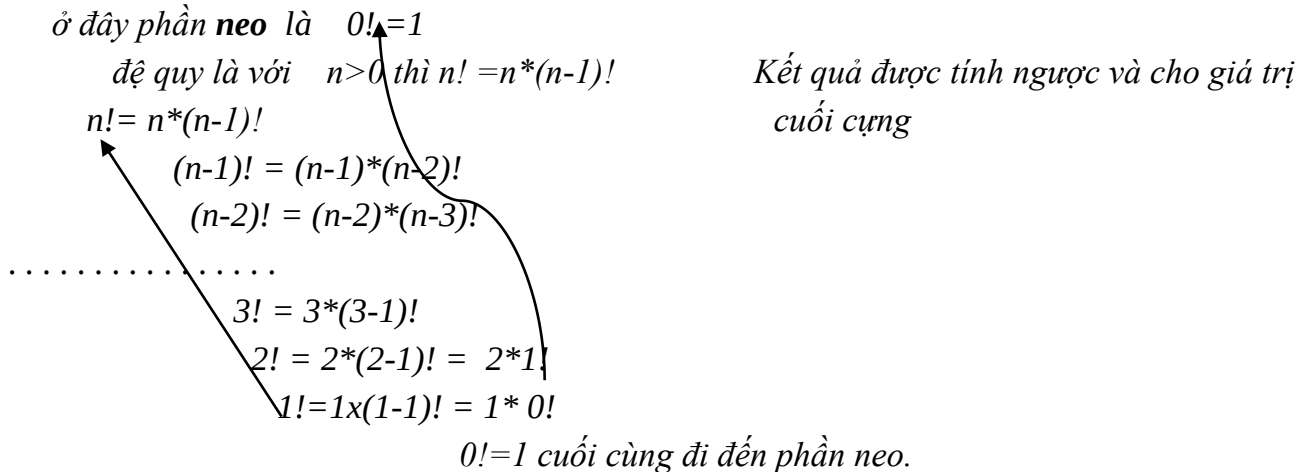
## 5.2. PHÉP ĐỆ QUY

Đệ quy là hiện tượng một hàm tham trỏ đến chính nó.

Một hàm đệ quy gồm hai phần:

1. **Phần neo** (anchor – cỏi neo dưng neo thuyền, tàu bố) trong đó tác động của hàm được đặc tả cho một hay nhiều tham số.
2. **Phần đệ quy**, trong đó tác động cần được thực hiện cho giá trị hiện thời của các tham số được định nghĩa bằng các tác động hay giá trị được định nghĩa trước đó.

Ví dụ 1: để tính giai thừa có thể mô tả đệ quy :



Viết hàm đệ quy sau :

```
int giaithua(int n)
{
    if(n==0)
    {
        return 1 ;
    }
    return n*giaithua(n-1);
}
```

ví dụ 2: số fibonacci

1

1

2

3

5

8

với  $n > 2$  mỗi số sau là tổng của hai số tr-ớc nó

Neo  $\begin{cases} \text{Fib}(1)=1 \\ \text{Fib}(2)=1 \end{cases}$

Đệ quy : với  $n > 2$ ,  $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

```
int Fib(int n)
{
    if(n<=2)
    {
        return 1;
    }
    return (Fib(n-1)+ Fib(n-2));
}
```

```

#include <iostream>
using namespace std;
int giaithua(int n)
{ if(n==0)
  { return 1 ;}
  return n*giaithua(n-1);
}

int Fib(int n)
{ if(n<=2)
  {return 1;}
  return (Fib(n-1)+ Fib(n-2));
}

int main()
{ cout << " \n\n    DE QUY - RECURSIVE \n\n" << endl;
  int n;
  //float hesoNewton;
  cout<<"    Nhap n :";cin>>n;
  cout<<"\n    Giai thua cua "<<n<<"! = "<<giaithua(n);
  cout<<"\n\n";
  cout<<"\n    So fibonacci cua "<<n<<" = "<<Fib(n);
  cout<<"\n\n";
  system("pause");

  return 0;
}

```

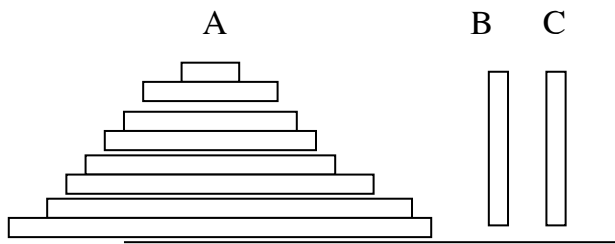
### **Bài tập**

- 1) Viết chương trình C++ dùng đệ quy tính USCLN của hai số nguyên.
- 2) Viết chương trình C++ dùng đệ quy tính hệ số nhị thức NewTon

$$C_{nk}^n = \frac{N!}{K!(N-K)!}$$

Ví dụ về phép đệ quy :

Tháp Hà nội



Cần phải chuyển các đĩa kích thước khác nhau từ cọc bên trái sang cọc bên phải, với các luật sau:

1. Khi di chuyển một đĩa, nó phải được đặt vào một trong ba cọc đã cho
2. Mỗi lần chỉ di chuyển một đĩa và phải là đĩa ở trên cùng
3. Đĩa lớn hơn không được phép nằm trên đĩa nhỏ hơn. Cho  $n$  là số đĩa.

Ta làm đệ quy như sau :

Với  $n=1$ , việc chuyển đĩa từ  $A \rightarrow C$  là giải được.

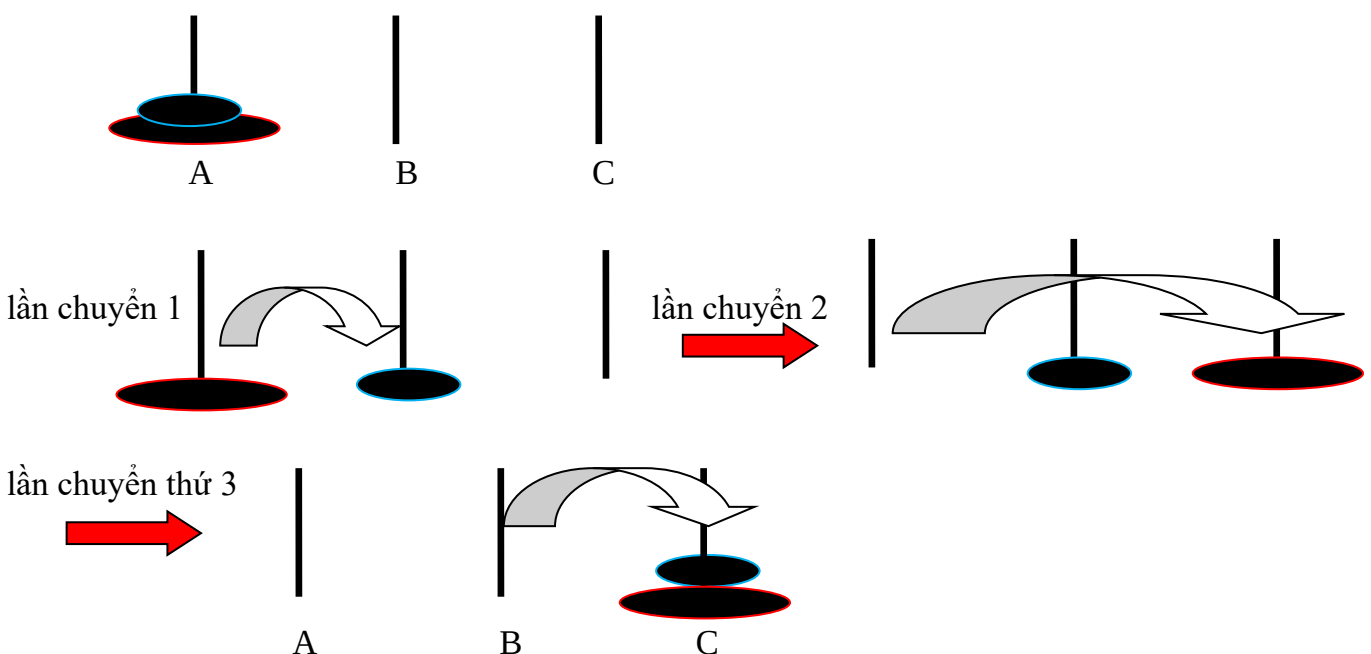
- 1, Giả sử đã chuyển được đối với  $n-1$  đĩa trên cùng ở  $A \rightarrow B$
2. Chuyển đĩa thứ  $n \rightarrow$  cột  $C$  ( $C$  là cọc phụ)
3. Chuyển  $n-1$  đĩa từ  $B \rightarrow C$ , dùng  $A$  là cọc phụ.

Ta cần chứng minh mệnh đề : bài toán tháp Hà nội chuyển  $n$  đĩa từ cọc  $A$  sang  $C$  có số lần chuyển là  $2^n - 1$ .

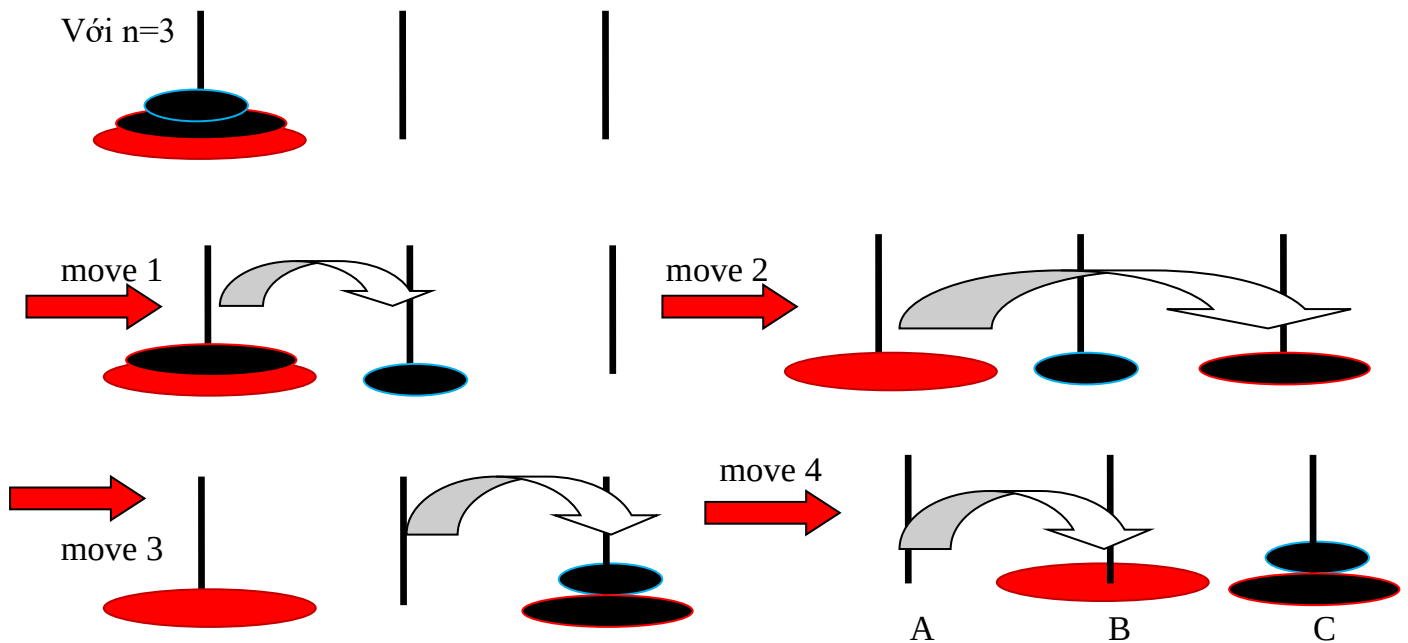
Chứng minh theo quy nạp.

Với  $n=1$ , việc chuyển đĩa từ  $A \rightarrow C$  số lần chuyển là 1

Với  $n=2$ , việc chuyển đĩa từ  $A \rightarrow C$  là :



Với 2 đĩa số lần chuyển là  $2^2 - 1 = 3$

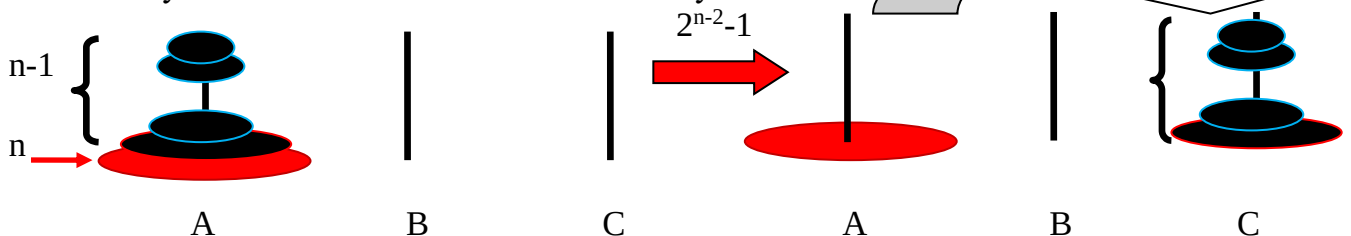


Từ cột C, chuyển 2 đĩa về B thì như đã xét sẽ cần 3 lần chuyển, vậy tổng số lần chuyển 3 đĩa là  $7 = 2^3 - 1$

...

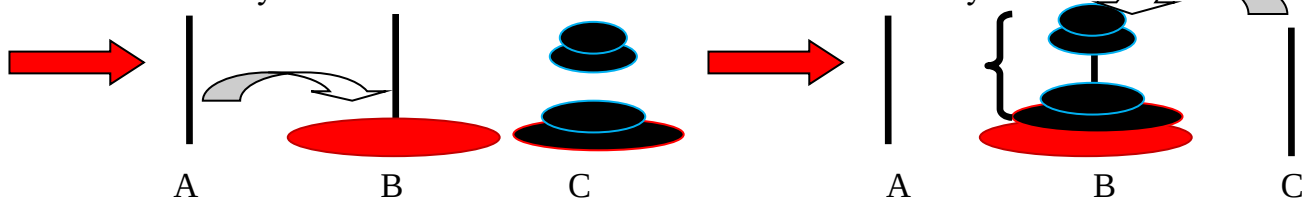
Giả thiết số lần chuyển  $n-1$  đĩa là  $2^{n-2} - 1$ , ta chứng minh công thức này đúng với  $n : 2^{n-1} - 1$ .

Phần 1 chuyển  $n-1$  đĩa từ A  $\rightarrow$  C cần  $2^{n-2} - 1$  lần chuyển



Tiếp theo chuyển đĩa thứ  $n$  sang cột B.  
1 lần chuyển

Phần cuối cùng chuyển  $(n-1)$  đĩa từ C  $\rightarrow$  B  
cần  $2^{n-2} - 1$  lần chuyển



Tổng số lần chuyển là  $(2^{n-2} - 1) + 1 + (2^{n-2} - 1) = 2 * (2^{n-2} - 1) + 1 = 2^{n-1} - 1$

Chương trình : `#include <iostream>`

`using namespace std;`

`void Move(int n, char Dau, char Phu, char Cuoi)`

```
{ if(n==1)cout<<"  Chuyen dia tu coc "<<Dau<<" sang coc "<<Cuoi;
  else { Move(n-1,Dau,Cuoi,Phu);cout<<"\n";
        Move(1,Dau,' ',Cuoi);cout<<"\n";
        Move(n-1,Phu,Dau,Cuoi);cout<<"\n";
      } }
```

```
int main()
{ cout << "\n\n\tTHAP HA NOI \n\n" << endl;
  const char coc1 ='A',coc2 ='B',coc3 ='C';
  int sodia; cout<<"\tNhap so dia : ";cin>>sodia;
  Move(sodia,coc1,coc2,coc3);
  system("pause");
  return 0; }
```

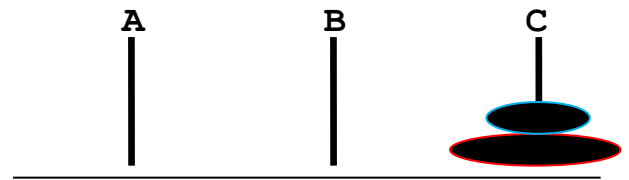
Kết quả chạy chương trình với số đĩa =2,3,4 (sinh viên tự lấy các đĩa ra thực nghiệm):

```

THAP HA NOI

Nhap so dia : 2
Chuyen dia tu coc A sang coc B
Chuyen dia tu coc A sang coc C
Chuyen dia tu coc B sang coc C
Press any key to continue . . .

```



```

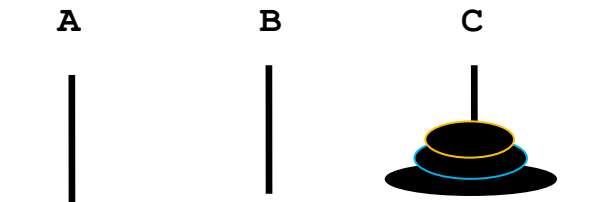
THAP HA NOI

      Nhap so dia : 3
Chuyen dia tu coc A sang coc C
Chuyen dia tu coc A sang coc B
Chuyen dia tu coc C sang coc B

Chuyen dia tu coc A sang coc C
Chuyen dia tu coc B sang coc A
Chuyen dia tu coc B sang coc C
Chuyen dia tu coc A sang coc C

Press any key to continue . . .

```



```

Nhap so dia : 4
Chuyen dia tu coc A sang coc B
Chuyen dia tu coc A sang coc C
Chuyen dia tu coc B sang coc C

Chuyen dia tu coc A sang coc B
Chuyen dia tu coc C sang coc A
Chuyen dia tu coc C sang coc B
Chuyen dia tu coc A sang coc B

Chuyen dia tu coc A sang coc C
Chuyen dia tu coc B sang coc C
Chuyen dia tu coc B sang coc A
Chuyen dia tu coc C sang coc A

Chuyen dia tu coc B sang coc C
Chuyen dia tu coc A sang coc B
Chuyen dia tu coc A sang coc C
Chuyen dia tu coc B sang coc C

Press any key to continue . . .

```

